



Figure 1: Puppet's communication with client servers

If you want to run it manually, you can run the command given below from the client server.

```
#puppet agent -test
```

So now, we have confirmed that Puppet is running fine and the master server is able to communicate with the client. Next, we will consider a few important files and folders.

The configuration file for open source Puppet is `/etc/puppet/puppet.conf`.

This *puppet.conf* file is divided into three parts, which are:

Main: Global configuration settings should be placed here.

Agent/user: This is to apply configurations to client serv

Master: These are configurations of the master server.
To list all the configurations by the command line, use the following code:

```
# puppet config print all
/var/log/puppet ==> contains puppet logs on both master and
client servers
/var/lib/puppet ==> contains Puppet operational data
/var/lib/puppet/ssl ==> contains SSL certificate
/etc/puppet/modules and /usr/share/puppet/modules ==> The
default directories where modules are searched in master
server
```

Let us create a simple file in `/home/user1/abc.txt` which contains “this is my first program” using Puppet.

In the screenshot shown in Figure 2, the *ensure* command checks the availability of the file/folder and, in case it is absent, it will create the respective file or folder. So it makes your job easy, because some apps may fail due to the absence of log files.

The directory attribute ensures `/home/user1` is a folder and is available. The `ensure` file attribute ensures the availability of a file; otherwise, it will create a file with the mentioned content.

By default, Puppet has many resource types like files, packages, services, file systems, etc. With reference to Figure 2, the file is the resource type and `/home/user1/abc.txt` is the

```
root@localhost:~# puppet apply -e 'file["/home/user1":
> ensure => "directory"
> file {"/home/user1/abc.txt":
> ensure => "present",
> content => "This is my first program",
> require => File["/home/user1"]; }'
Notice: Compiled catalog for localhost.puppetlabs.vm in environment production in
0.07 seconds
root@localhost:~# cat /home/user1/main/file["/home/user1"]/.ensure: created
root@localhost:~# cat /home/user1/main/file["/home/user1/abc.txt"]/.ensure: defined content as
"this is my first program"
root@localhost:~# cat /home/user1/main/file["/home/user1/abc.txt"]/.content:
this is my first program
root@localhost:~#
root@localhost:~# ll -l /home/user1/abc.txt
-rw-r--r-- 1 root root 24 Sep 13 14:15 /home/user1/abc.txt
root@localhost:~# cat /home/user1/abc.txt
This is my first program
root@localhost:~#
```

Figure 2: Example of a Puppet program

attribute. The power of Puppet is that we can create our own resource types.

To view the resource types available, use the following code:

```
#puppet resource -types
computer
cron
docker_compose
docker_network
exec
file
file_line
group
host
interface
maillist
mount
nagios_command
<Truncated>
```

Puppet also has modules like pluggable libraries in applications. These modules are groups of manifests and data such as facts, files and templates. Modules are designed with different components per system. Readymade manifests are available in different scopes such as run instance, image management, repository management and service management.

As an example, the directory base for the Apache module is:

<MODULE_PATH>httpd/

A directory in this Apache module that serves static files is:

<MODULE_PATH>/httpd/files

A static configuration file for *httpd* is shown below:

```
<MODULE_PATH>/httpd/files/myhttpd.conf
AccessFileName .acl
```